



African Digital Logistics & Infrastructure Simulator

Project Report: WAS262 Web Animation Scripting (SF1)

Student name	Shivay Goorchuran
Student number	25301767
GitHub repository	EcoDash-African-Logistics: https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-
Live application	https://shivaygoorchuran94-droid.github.io/EcoDash-African-Logistics-/

Contents

Contents

- 1. Introduction and project overview 4
 - 1.1 Requirement-to-implementation summary 4
- 2. Task 1.1: GitHub repository setup 4
 - Installation and setup 5
 - Commit history evidence 5
- 3. Task 1.2: African problem investigation and mathematical model 6
 - 3.1 Problem context 6
 - 3.2 Real-world factor to game-mechanic mapping 6
 - 3.3 Mathematical model..... 6
 - Worked examples. 7
- 4. Task 1.3: Wireframe and AI reflection 7
 - 4.1 Wireframe: main game screen 7
 - 4.2 Wireframe: overlay screens..... 9
 - 4.3 AI Usage Disclosure Table 9
 - 4.4 AI Reflection Log 11
 - Ethical concerns 11
- 5. Task 2: Game development (implementation) 11
 - 5.1 Architecture, game loop, and state machine..... 11
 - 5.2 Player movement and physics (2.1)..... 12
 - 5.3 Obstacles and collision detection (2.2)..... 13
 - 5.4 User experience and the original feature (2.3)..... 14
 - Sound 14
 - Visual theme and HUD 14
 - Responsive interface..... 14
 - Animation principles demonstrated 15
 - Original feature: procedural day/night cycle with parallax (authored without generative AI) 15
 - 5.5 Score system and local storage (2.4) 16
- 6. Testing, known limitations, and code-defence notes 17
 - 6.1 Test plan 17

6.2 Known limitations and future work.....	18
6.3 Code-defence quick reference.....	18
7. Conclusion	18
References	19

1. Introduction and project overview

EcoDash is an interactive two-dimensional simulation built for the WAS262 Web Animation Scripting module (STADIO, 2026). The player pilots a **solar-powered delivery drone** across an African savanna, collects medical cargo pods, avoids infrastructure and wildlife hazards, and manages a battery that is drained by flight and weather and recharged only at **Solar Microgrid Zones**, which shut down whenever the simulated grid enters **load-shedding**.

The application uses only HTML5 Canvas, CSS3 and vanilla JavaScript (ES6+), with no engine or framework, as required by the brief (STADIO, 2026). Rendering uses the Canvas 2D API (Mozilla, n.d.a), audio is synthesised with the Web Audio API (Mozilla, n.d.b), and scores persisted with `localStorage` (Mozilla, n.d.c). The code base is about 3 400 lines across five JavaScript modules, one stylesheet and one HTML page.

1.1 Requirement-to-implementation summary

Brief requirement	How EcoDash meets it	Where in the code
2.1 Movement and physics	Velocity, acceleration, gravity, thrust, linear drag, velocity cap, sine/cosine-driven effects, battery system, OOP entity classes	Drone in <code>entities.js</code> ; constants in <code>Config.player</code>
2.2 Obstacles and collision	Pylons, baobab trees, cranes, birds, and storm cells; circle-circle and circle-AABB tests; weather and load-shedding change gameplay	<code>Game.handleInteractions</code> , <code>testCollision</code> , <code>Utils.circleRect</code>
2.3 UX and original feature	Procedural music and SFX, savanna theme, canvas HUD, responsive layout; day/night cycle with four-layer parallax	<code>engine.js</code> (Audio), <code>game.js</code> (HUD), <code>world.js</code> (Background), <code>style.css</code>
2.4 Score and storage	Score, distance, energy-efficiency score, top-5 high scores, start, pause and game-over screens, restart without reload	Storage in <code>config.js</code> ; <code>Game.gameOver</code> , <code>resetRun</code>

2. Task 1.1: GitHub repository setup

The project is kept in a public repository named **EcoDash-African-Logistics**, as specified by the brief. The `README.md` contains the project title, description, installation and setup instructions, the AI Usage Disclosure Table (Section 4.3), and the folder structure below. The structure matches the paths referenced in `index.html` (`css/style.css` and `js/*.js`).

```
EcoDash-African-Logistics/  
|-- index.html      (canvas + DOM overlay screens, loads scripts)  
|-- README.md       (title, description, setup, AI disclosure, original feature)  
|-- css/  
|  \-- style.css    (layout shell, menus, responsive rules)  
\-- js/  
    |-- config.js    (Config, Utils, Storage)  
    |-- engine.js     (Audio, Input, Particle, ParticleSystem)  
    |-- world.js      (Background: day/night + parallax, Weather)  
    |-- entities.js   (Entity, Drone, Pylon, Baobab, Crane, Bird,
```

| StormCell, CargoPod, SolarZone)
 \-- game.js (HUD, Game state machine, bootstrap)

Installation and setup

- Clone or download the repository: `git clone` the public URL or download the ZIP from GitHub.
- Open `index.html` in a modern browser (Chrome, Edge, or Firefox). No build step or dependencies are needed.
- Optional: run through the VS Code Live Server extension so the page is served over `http://localhost`.
- Deployment: in the repository, choose Settings, then Pages, then deploy from the main branch root to obtain the live link.

Script loading order matters because each file extends the shared window. EcoDash namespace: `config.js`, `engine.js`, `world.js`, `entities.js`, then `game.js`.

Commit history evidence

The brief requires at least eight meaningful commits on different days. The table below is the planned milestone sequence that follows the five code layers; the dates and commit hashes must be filled in from the real GitHub history.

#	Date	Commit message (suggested)	Evidence
1	17/08/2026	Initial commit: README	https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-/commit/76f1eee452202f2d7e6f1df7d3bcf3c52e3e8097
2	18/08/2026	index.html and canvas setup	https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-/commit/5ad34b5c27772a53cd4b6756d2cbdd21319cf149
3	19/08/2026	Add battery system, HUD, and game loop	https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-/commit/31390fd992b0e4eb86fb83cebc32da3f1a6577
4	20/08/2026	Add obstacles, spawner, and collision detection	https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-/commit/7c6c42e2f748e2c2dd7f3128cd4466b8e0dde6da
5	21/08/2026	Add Web Audio music and SFX, particles, screen states, high scores	https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-/commit/39117cb85368cd89342096904c1681ac7136c3e5
6	22/08/2026	Add Weather, load-shedding and solar zones	https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-/commit/03b6f9d7116f11cfb7775e36a37a49466b5487dd
7	24/08/2026	Add Background: parallax layers and day/night cycle	https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-/commit/31ae740ef6bfed8db4e59b065558eba4f28eb2c0
8	25/08/2026	responsive CSS	https://github.com/shivaygoorchuran94-droid/EcoDash-African-Logistics-/commit/e91ef76b2810c97c1a65ea6f886a41c6117f624f

3. Task 1.2: African problem investigation and mathematical model

3.1 Problem context

Rural and underserved communities across Africa face two connected infrastructure problems: unreliable transport links and unreliable electricity. EcoDash models both together.

Last-mile medical delivery. In Rwanda, a hilly, mostly rural country, the government contracted Zipline in 2016 to deliver emergency blood by drone, and drones now carry many kinds of medical supplies to clinics that are hard to reach by road (NPR, 2026). The same reporting describes a woman who needed three hours to reach a doctor, which shows why delivery time matters clinically. Drones bypass winding roads, but they must still cope with weather, obstacles, and energy limits, which are the pressures the game turns into gameplay.

Load-shedding. In South Africa, Eskom cuts supply to parts of the grid in controlled stages when generation cannot meet demand, and each stage corresponds to another 1 000 MW removed, from Stage 1 up to Stage 8 (Daily Investor, n.d.). For a solar-charged logistics network this matters because charging stations, depots and communications can lose grid support at short notice. EcoDash therefore includes load-shedding events (Stage 2 to Stage 6) that take the microgrid charging zones offline.

Design problem statement: how can a solar-powered delivery drone keep medical cargo moving through hostile weather and infrastructure hazards when charging opportunities are intermittent? The simulation lets the player experience the trade-off between flying aggressively (fast, high-energy use) and flying efficiently (slower, energy-aware).

3.2 Real-world factor to game-mechanic mapping

Real-world factor	Game mechanic	Implementation
Medical supplies and clinics	Cargo pods: every 5 pods complete a delivery with a bonus	CargoPod, collectCargo, podsPerDelivery = 5
Solar microgrids	Zones that recharge the drone while it is inside them	SolarZone, Drone.inSolar, solarRegen
Load-shedding	Timed grid-offline events (Stage 2 to 6) that disable recharging and show a warning banner	Game.updateLoadShedding, loadShedding.active
Crosswinds	Vertical wind acceleration that oscillates over time	Weather.windyY, scaled by intensity for every weather type, added to acceleration
Dust storms and rain	Extra battery drain in every non-clear weather type, plus reduced visibility in dust and rain (tint, haze, vignette, particles)	drainWeather, Weather.draw
Power lines, cranes, trees, birds, storms	Collision hazards that end the mission	Pylon, Crane, Baobab, Bird, StormCell
Route length and time pressure	Scrolling speed increases with distance travelled	scrollSpeedStart to scrollSpeedMax

3.3 Mathematical model

The world scrolls horizontally at `worldSpeed` (in pixels per second) while the drone controls its own vertical motion. The player therefore experiences a velocity vector whose horizontal component is the scroll speed and whose vertical

component is the drone velocity. Motion follows the standard kinematic relations between position, velocity, and acceleration (Halliday, Resnick and Walker, 2014), integrated each frame with a time step dt seconds.

Quantity	Formula used	Constants (Config)
Net vertical acceleration	$a = g + T \cdot [\text{thrust}] + D \cdot [\text{descend}] + w(t)$	$g = 640, T = -1250, D = 780 \text{ (px/s}^2\text{)}$
Velocity update	$v \leftarrow v + a \cdot dt; v \leftarrow v - k \cdot v \cdot dt; v \leftarrow \text{clamp}(v, -v_{\text{max}}, v_{\text{max}})$	$k = 3.4 \text{ (drag)}, v_{\text{max}} = 430 \text{ px/s}$
Position update	$y \leftarrow y + v \cdot dt$	dt capped at $1/30 \text{ s}$
Crosswind	$w(t) = \sin(\phi) \cdot F \cdot I, \phi \leftarrow \phi + dt \cdot (0.75 + 1.7 \cdot I)$	$F = 165 \text{ px/s}^2, I = \text{intensity } 0 \text{ to } 1$
Tilt angle (visual)	$\theta \leftarrow \theta + (\theta_{\text{target}} - \theta) \cdot \min(1, 8 \cdot dt), \theta_{\text{target}} = 0.42 \cdot \text{clamp}(v/v_{\text{max}}, -1, 1)$	max tilt 0.42 rad (about 24°)
Battery change	$\Delta B = -(d0 + dt_h \cdot [\text{thrust}] + dw \cdot I \cdot [\text{weather} \neq \text{clear}]) \cdot dt + r \cdot dt \cdot [\text{in solar zone and grid on}]$	$d0 = 1.05, dt_h = 2.45, dw = 0.85, r = 90 \text{ (\% per s)}$
Distance	$\Delta \text{metres} = \text{worldSpeed} \cdot dt / 6$	6 px per metre
Difficulty	$\text{difficulty} = \text{clamp01}(\text{distance} / 2200); \text{speed} = \text{lerp}(152, 436, \text{difficulty})$	scroll $152 \text{ to } 436 \text{ px/s}$
Energy efficiency	$\text{efficiency} = \text{distance} / \max(1, \text{totalDrain})$	metres per battery percent

Worked examples.

Because drag is proportional to velocity, each input has a terminal velocity where acceleration equals drag ($a = k \cdot v$):

- **Free fall:** $640 / 3.4 = \text{about } 188 \text{ px/s downward.}$
- **Full thrust:** $(640 - 1250) / 3.4 = \text{about } -179 \text{ px/s, i.e. climbing at } 179 \text{ px/s.}$
- **Dive:** $(640 + 780) / 3.4 = \text{about } 418 \text{ px/s, just under the } 430 \text{ px/s cap.}$
- **Peak crosswind:** $165 / 3.4 = \text{about } 49 \text{ px/s of extra drift, which the player must correct with thrust.}$

Energy budgets follow directly: idle flight drains 1.05% per second (about 95 s from full), thrusting drains 3.5% per second (about 29 s), and a full-intensity storm adds up to 0.85% per second. Inside an active solar zone, the net change while idling is $90 - 1.05$, about $+89 \%$ per second. At the starting scroll speed a pass through a zone lasts roughly $1.2 \text{ to } 1.6 \text{ s}$, which refills an empty battery; at maximum speed a pass lasts only about $0.4 \text{ to } 0.6 \text{ s}$ and restores roughly $40 \text{ to } 50 \%$. Recharging works only when load-shedding is off.

Trigonometry is used throughout: `Math.sin` drives crosswind, bird flight paths ($y = \text{baseY} + \sin(\text{age} \cdot \text{freq} + \text{phase}) \cdot \text{amp}$), cargo-pod arcs ($y = \text{baseY} + \sin(i \cdot 0.85) \cdot \text{amp}$), the sun and moon arcs, rotor blur and sway; `Math.cos` drives the night-darkness value (Section 5.4); and `Math.cos/Math.sin` convert polar to Cartesian velocity in `ParticleSystem.burst` ($v_x = \cos(\text{angle}) \cdot \text{speed}, v_y = \sin(\text{angle}) \cdot \text{speed}$).

4. Task 1.3: Wireframe and AI reflection

4.1 Wireframe: main game screen

The wireframe below shows the layout of the running simulation. Numbered markers are explained in Table 1. Green markers identify the original feature (Section 5.4).

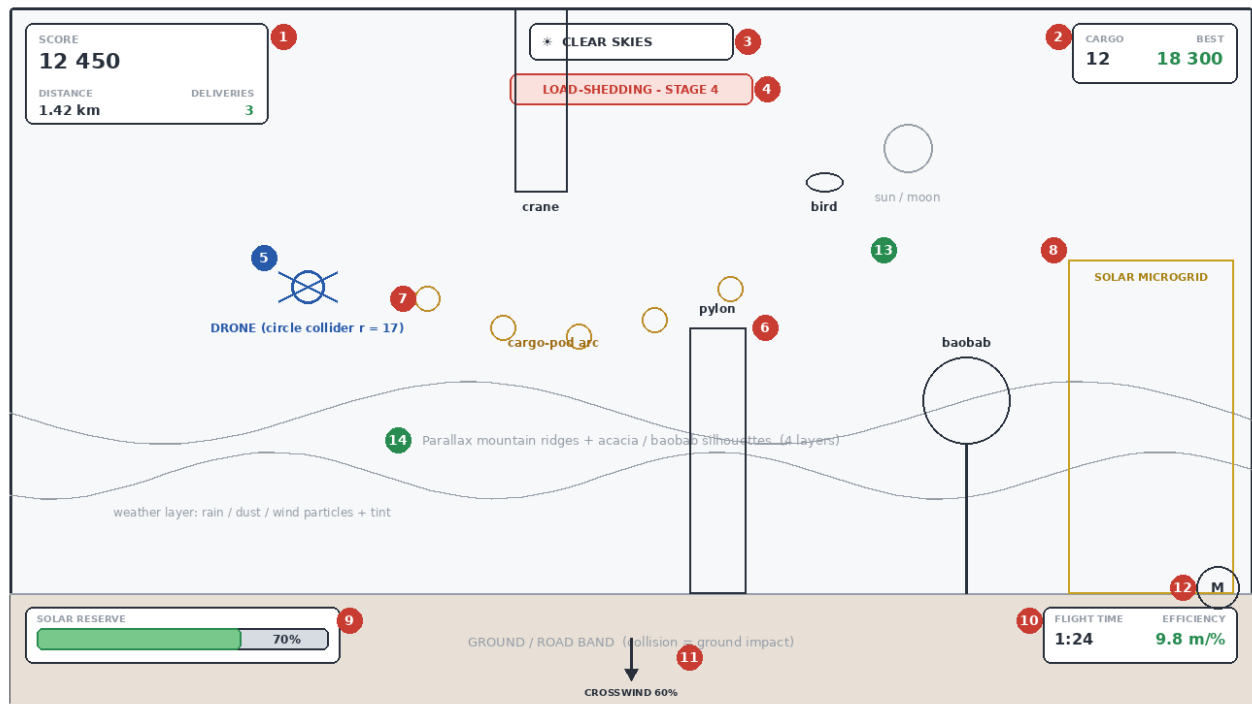


Figure 1: Wireframe of the main game screen with numbered HUD and gameplay elements.

No.	Element	Purpose and behaviour
1	Score panel	Live score, distance travelled and deliveries completed (top left)
2	Cargo and best score	Pods collected this run and the saved best score (top right)
3	Weather chip	Current weather (clear, crosswind, dust storm, heavy rain) with an intensity meter
4	Load-shedding banner	Flashing red banner with the stage number while the grid is offline
5	Player drone	Circle collider, radius 17 px; tilts with vertical velocity; rotors spin faster when thrusting
6	Obstacles	Pylons, baobab trees, cranes, birds, storm cells; any contact ends the mission
7	Cargo pod arc	Collectables laid out on a sine curve; 50 points each
8	Solar microgrid zone	Recharges the battery unless load-shedding is active
9	Solar reserve bar	Battery percentage, colour-coded green, amber, red, pulsing at 25 % or lower
10	Efficiency and flight time	Energy-efficiency score in metres per percent, and elapsed time
11	Wind indicator	Arrow showing crosswind direction and strength
12	Mute button	Toggles all audio (also the M key); preference is saved
13	Sky, sun, moon, stars	Original feature: day/night cycle (about 115 s per full day)
14	Parallax landscape	Original feature: four-layer scrolling ridges and trees at different speeds

Table 1: Wireframe legend.

4.2 Wireframe: overlay screens

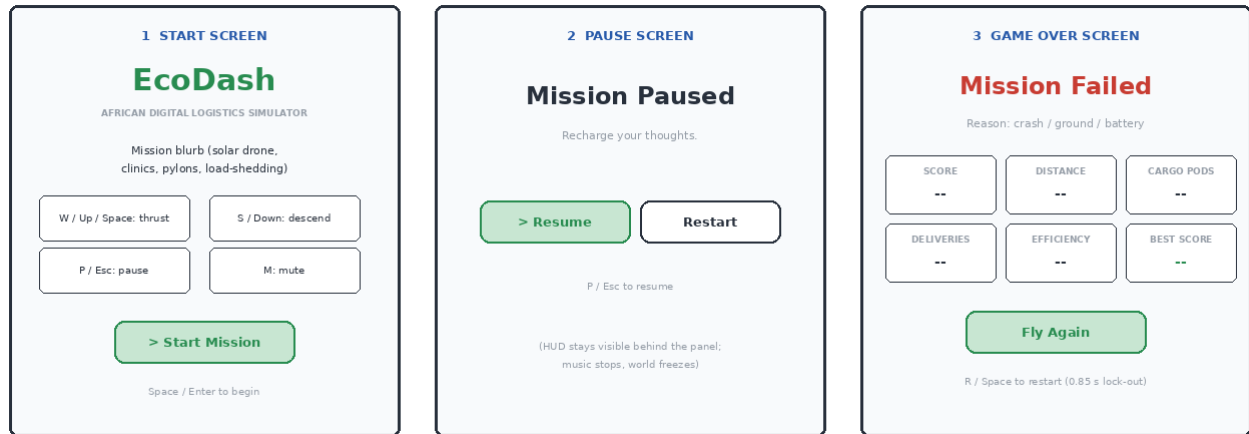


Figure 2: Start, pause and game-over screens (DOM overlays above the canvas).

4.3 AI Usage Disclosure Table

This table records where generative AI was and was not used, as required by the AI Traffic Light Matrix (amber: AI may be used to assist, with evidence of prompts and outputs). The world.js header states that the background day/night and parallax feature was authored without generative AI. All other cells must be completed truthfully by the student from their own prompt history.

Component	AI used?	Tool and purpose	What I changed or can explain
Day/night cycle and parallax (Background)	No	None	Sky keyframes, $\text{night} = 0.5 + 0.5 \cdot \cos(2\pi t)$, parallax offsets (Section 5.4)
Drone physics	No	None	Each frame: acceleration = gravity + thrust (W/Up/Space/pointer) + dive (S/Down) + weather windY, then drag, clamp to maxVy, all scaled by dt. Ceiling bounce (x -0.25) and ground stop. Constants live in Config.player; solarRegen was raised from 34 to 90 %/s for a faster pit-stop.
Collision detection	No	None	Circle-vs-circle test (distance squared $\leq (r1+r2)^2$) for baobabs, birds and storm cells; circle-vs-rectangle using the clamped nearest point (Utils.circleRect) for pylons, cranes and SolarZones. Cargo uses a pickup radius 12 px larger than the hitboxes. Any obstacle hit ends the run; clearing one scores +15.
Weather and load-shedding	No	None	Weather picks clear 50%, crosswind 20%, dust 17%, rain 13%, easing intensity toward a target. A sine-wave windY pushes the drone and bad weather adds battery drain. Load-shedding toggles on a random timer (stage 2-6); while active, SolarZones do not recharge the drone.

Component	AI used?	Tool and purpose	What I changed or can explain
Web Audio music and SFX	No	None	Procedural Web Audio only, no audio files: oscillator tones and filtered noise for the SFX, plus a look-ahead scheduler looping a minor-pentatonic melody and bass. Separate music and SFX gain buses; mute is saved in preferences and the AudioContext unlocks on first input.
HUD and CSS	No	None	HUD is drawn on the canvas in game.js (weather chip, load-shedding banner, battery bar, efficiency, wind meter) and scales with screen size. style.css styles the DOM overlay panels using CSS variables, with responsive media queries and a reduced-motion rule.
Particle system and storage	No	None	Pooled particle system (max 520) that reuses inactive particles, with life, drag, gravity and growth, and a burst helper for pickups and exhaust. Storage keeps the top 5 scores in localStorage plus preferences, using try/catch and a safe JSON parse.
README and report drafting	No	None	EcoDash is an interactive HTML5 Canvas simulation where players control a solar-powered drone or electric vehicle delivering essential supplies across African terrain. Manage battery, weather, obstacles, load-shedding and complete missions. Built with vanilla JavaScript, CSS3, and OOP, it uses physics, collision detection, scoring, local storage.

4.4 AI Reflection Log

Ethical concerns

Academic integrity and code ownership. Submitting AI-generated code as original work breaches STADIO's plagiarism policy (STADIO, 2026). The safeguard is to disclose every use, write the original feature without generative AI, and be able to explain and modify every line live during the code defence.

Over-reliance and skill loss. Accepting suggestions without understanding them weakens the very skills the module assesses. AI output can also be subtly wrong (for example, frame-rate-dependent physics or hard-coded magic numbers), so each suggestion must be assessed, questioned, and adapted to the project rather than pasted in. A third concern is cultural accuracy: generic AI output can fall back on stereotypes when asked for an "African theme", so the context in this project is grounded in specific, verifiable challenges such as load-shedding and rural drone logistics (NPR, 2026; Daily Investor, n.d.). As for this project, no AI was used.

5. Task 2: Game development (implementation)

5.1 Architecture, game loop, and state machine

Modular structure. Each file is a layer, and each layer is an immediately-invoked function expression (IIFE) that attaches its public classes to a single window.EcoDash namespace. This avoids global variable clashes and keeps dependencies explicit (Flanagan, 2020).

File	Public objects	Responsibility
config.js	Config, Utils, Storage	All tuning constants; maths, colour, and drawing helpers; localStorage persistence
engine.js	Audio, Input, Particle, ParticleSystem	Procedural sound; keyboard and pointer input; pooled particle effects
world.js	Background, Weather	Sky, day/night, parallax, ground; weather types, wind, and effects
entities.js	Entity, Drone, obstacle, and pickup classes	Game objects with update and draw methods
game.js	HUD, Game, bootstrap	State machine, spawning, collisions, scoring, rendering, DOM wiring

Object-oriented design. Entity is the base class holding position, velocity, collider shape, and lifecycle flags. Drone, Pylon, Baobab, Crane, Bird, StormCell, CargoPod and SolarZone inherit from it using `Object.create(Entity.prototype)`, and override `update` and/or `draw` (polymorphism). Because every entity exposes `shape`, `kind`, `x`, `y` and either `radius` or `w/h`, the Game class can treat them uniformly in one loop.

Game loop. `requestAnimationFrame` (Mozilla, n.d.d) drives a variable time-step loop: the elapsed time is converted to seconds and clamped to 1/30 s so a background-tab pause or lag spike cannot cause objects to tunnel through each other (Nystrom, 2014; Gregory, 2018). The loop calls `update(dt)` and then `draw()`.

```
Game.prototype.loop = function (now) {  
  this._rafId = requestAnimationFrame(this._boundLoop);  
  var dt = (now - this._lastTime) / 1000;
```

```

this._lastTime = now;
if (!isFinite(dt) || dt < 0) dt = 0;
dt = Math.min(dt, 1 / 30); // guard against tab-switch spikes
this.update(dt);
this.draw();
};

```

State machine. `Game.state` is one of `menu`, `playing`, `paused` or `over`. `setState` fires a callback that toggles the matching DOM overlay and sets data-state on the body. Only the `playing` state advances physics, spawning, and scoring, while the `menu` runs an idle hover with a slow scrolling background. The game auto-pauses when the tab is hidden.

Input. Input distinguishes a held key (down) from a key pressed this frame (pressed), which is cleared by `endFrame()`. Held state gives thrust and descend; pressed state triggers one-shot actions such as pause. It reads `e.code`, so controls are layout independent (W, arrow up or Space to thrust; S or arrow down to descend). A pointer or touch on the canvas acts as thrust, and losing window focus clears all keys so the drone cannot get stuck thrusting.

5.2 Player movement and physics (2.1)

The Drone class implements the model in Section 3.3. Each frame it builds a net vertical acceleration from gravity, thrust, dive and wind, integrates velocity with drag and clamping, then moves and applies boundary rules: a soft ceiling that reflects 25 % of upward speed, and a ground that ends the mission.

```

var accel = p.gravity;           // 640
if (this.thrusting) accel += p.thrust; // -1250
if (this.diving)  accel += p.dive;    // +780
accel += game.weather.windY;        // sin-based crosswind

this.vy += accel * dt;
this.vy -= this.vy * p.drag * dt;    // linear drag
this.vy = U.clamp(this.vy, -p.maxVy, p.maxVy);
this.y += this.vy * dt;

```

Smooth, directional motion. The drone tilts towards its direction of travel using exponential easing, `angle += (targetAngle - angle) * Math.min(1, dt * 8)`, so the nose leads the movement rather than snapping. Rotor speed rises from 26 to 44 rad/s when thrusting, giving immediate visual feedback for input. Both effects are frame-rate independent because they scale with `dt`.

Battery (Solar Reserve). Drain has three components: a constant base, extra drain while thrusting, and extra drain scaled by weather intensity. Recharging occurs only when the drone overlaps a `SolarZone` and load-shedding is inactive. The battery is clamped to the range 0 to 100, a low-battery beep sounds at 25 % or below, and a battery of 0 ends the mission.

```

var drain = p.drainBase;           // 1.05 %/s
if (this.thrusting) drain += p.drainThrust; // +2.45 %/s

```

```

if (game.weather.type !== 'clear')
  drain += p.drainWeather * game.weather.intensity;    // up to +0.85 %/s
this.battery -= drain * dt;
this.totalDrain += drain * dt;                          // used for efficiency
if (this.inSolar && !game.loadShedding.active)
  this.battery = Math.min(p.batteryMax, this.battery + p.solarRegen * dt);

```

Descriptive naming. Names describe purpose (solarRegen, drainThrust, cargoSinceDelivery, overCooldown, worldSpeed, totalDrain) rather than single letters, and the main gameplay tuning constants live in `EcoDash.Config`, which also makes live changes during the code defence quick and safe. A few presentation values sit beside the code that uses them, for example `CYCLE_SECONDS` in `world.js` and the 15-point pass bonus in `Game.handleInteractions`.

5.3 Obstacles and collision detection (2.2)

Obstacles spawn off-screen to the right and scroll left at `worldSpeed`; the spawner picks a feature type by weighted random roll each time the spawn timer expires. New obstacle types unlock as difficulty rises, and the gap between spawns shrinks from 1.30 to 2.10 s at the start to 0.72 to 1.15 s at maximum difficulty.

Obstacle	Collider	Unlocks at difficulty	Behaviour
Pylon (transmission tower)	Rectangle (AABB)	Start	Ground-based lattice tower with a blinking red aircraft-warning light; height 24 to 54 % of the ground line
Baobab tree	Circle (canopy)	Start	Round canopy on a trunk that sways with a sine wave
Construction crane	Rectangle	> 0.12	Hangs from the top of the screen with a swinging hook
Bird (wildlife)	Circle	> 0.25	Flies faster than the scroll speed with a sinusoidal vertical path and flapping wings
Storm cell	Circle	> 0.42	Large cloud with lightning flashes that also flash the whole screen

Collision maths. Circular colliders use squared distance so no square root is needed: two circles overlap when $dx^2 + dy^2 \leq (r_1 + r_2)^2$. For rectangles, the drone circle is assessed against an axis-aligned box by clamping the circle centre to the box to find the nearest point, then comparing squared distance with r^2 (Ericson, 2005). Any obstacle hit triggers `gameOver("crash")`.

```

Game.prototype.testCollision = function (drone, e) {
  if (e.shape === 'circle') {
    var rr = e.radius + drone.radius;
    return U.dist2(drone.x, drone.y, e.x, e.y) <= rr * rr;
  }
  return U.circleRect(drone.x, drone.y, drone.radius, e.x, e.y, e.w, e.h);
};

```

```

function circleRect(cx, cy, r, rx, ry, rw, rh) {

```

```

var nx = clamp(cx, rx, rx + rw); // nearest point on the box
var ny = clamp(cy, ry, ry + rh);
return dist2(cx, cy, nx, ny) <= r * r;
}

```

Other interactions. Cargo pods use a slightly generous pickup radius (pod radius + drone radius + 12 px) so collecting feels fair. Solar zones use the same circle-rectangle test but do not harm the player. Hitting the ground ends the mission with reason ground, and an empty battery ends it with reason battery.

Environmental conditions change gameplay. Weather intensity scales a time-varying vertical wind force that pushes the drone up and down, strongest in a crosswind but present in dust and rain as well. All three non-clear weather types (crosswind, dust storm, heavy rain) increase battery drain, while dust and rain also reduce visibility through tinting, haze, vignetting, and particles. Load-shedding switches every solar zone offline (greyed panels and an "OFFLINE" label) and blocks recharging. Weather is chosen randomly (50 % clear, 20 % wind, 17 % dust, 13 % rain) and fades in and out with an exponential approach so changes are gradual.

Passing obstacles. When an obstacle has fully passed the drone without contact, a horn sound plays and 15 points are awarded once (e. passed prevents repeats), with a small green particle burst as feedback.

5.4 User experience and the original feature (2.3)

Sound

All audio is generated in code with the Web Audio API (Mozilla, n.d.b), so no external files are needed. A master gain node feeds a music bus (volume 0.14) and an effects bus (volume 0.5). Music is a looping melody on a minor-pentatonic set of semitones (0, 3, 5, 7, 10 above G3 at 196 Hz) with a bass line, scheduled about 0.25 s ahead by a 60 ms timer so playback stays in time even when the main thread is busy. Effects include a three-tone horn, pickup chime, delivery arpeggio, crash (filtered noise burst plus a falling sawtooth), load-shedding alarm, low-battery beep, and a rising charge sweep. Audio is unlocked on the first user gesture to satisfy browser autoplay rules, and the mute preference is saved.

Visual theme and HUD

The scene is a savanna with acacia and baobab silhouettes, warm laterite ground with a road band, a gold-and-green colour palette, and a drone carrying a medical cargo box. The HUD is drawn on the canvas and shows score, distance, deliveries, cargo, best score, weather, load-shedding status, battery, efficiency, flight time, and a wind arrow. HUD sizes are scaled by $s = \text{clamp}(\text{width} / 1280, 0.72, 1.35)$, so it stays legible on phones and large screens.

Responsive interface

The canvas fills the viewport and is resized with a debounced handler on `resize` and `orientationchange`; the backing store is scaled by `devicePixelRatio` (capped at 2) for sharp rendering. The CSS uses custom properties, `min()` and `clamp()` for fluid sizing, a media query for narrow screens (under 560 px), a landscape query for short viewports, `touch-action: none` on the canvas, and a `prefers-reduced-motion` rule. Buttons have visible focus outlines, and the canvas has an ARIA label.

Animation principles demonstrated

The brief asks for selected principles of animation to be demonstrated. EcoDash applies the following principles from Thomas and Johnston (1981):

Principle	Where it appears
Slow in and slow out	Weather intensity eases towards its target; drone tilt eases towards the velocity-based angle; particles fade out over the last 35 % of their life
Follow-through and overlapping action	The drone body tilts a moment after velocity changes; tree canopies and crane hooks sway independently
Arcs	Cargo pods follow sine arcs; birds fly sinusoidal paths; the sun and moon travel in arcs across the sky
Secondary action	Rotor blur, dust trail, blinking status LED and pylon light, lightning, solar sparkles
Exaggeration	Crash produces screen shake, an explosion of debris and sparks, and smoke
Staging	Glowing halos on pods, colour-coded battery bar, night headlight cone, and a pulsing load-shedding banner direct the player's attention

Original feature: procedural day/night cycle with parallax (authored without generative AI)

The original feature is the Background class in world.js, which combines a continuous day/night cycle with four parallax layers. It is intentionally independent of the gameplay code, so it can be explained and modified from first principles.

Time of day. A variable `timeOfDay` runs from 0 to 1 and wraps every 115 seconds (`CYCLE_SECONDS`). It starts at 0.30, which is early morning. A darkness value is derived from a cosine so night fades in and out smoothly instead of switching:

```
this.timeOfDay = (this.timeOfDay + dt / CYCLE_SECONDS) % 1;
this.night = 0.5 + 0.5 * Math.cos(this.timeOfDay * TAU);
// t = 0 -> night = 1 (midnight), t = 0.5 -> night = 0 (noon)
```

Sky colour. Nine colour keyframes (deep navy, purple dawn, blue day, orange dusk, and back) are stored as top and bottom gradient colours. `skyColors(t)` finds the two keyframes surrounding `t`, computes the local fraction $k = (t - a.t) / (b.t - a.t)$, and blends them with `lerpColor`. The blended pair becomes a vertical linear gradient.

Sun, moon, and stars. For the sun, $\text{dayT} = (\text{timeOfDay} - 0.25) / 0.5$ maps the daytime half of the cycle to 0 to 1; the horizontal position is $\text{lerp}(-0.08, 1.08, \text{dayT}) \cdot \text{width}$ and the height is $0.88 \cdot h - \sin(\pi \cdot \text{dayT}) \cdot 0.74 \cdot h$, which traces a half-sine arc. The moon follows the same half-sine arc shifted by half a cycle, with a slightly lower peak ($0.70 \cdot h$). 140 stars are generated once from a seeded random generator and drawn only when $\text{night} > 0.04$, with alpha $\text{night} \cdot \text{twinkle} \cdot 0.95$, where $\text{twinkle} = 0.55 + 0.45 \cdot \sin(t \cdot \text{speed} + \text{phase})$.

Lighting. A dark blue overlay with alpha $\text{night} \times 0.50$ is drawn over the world and a lighter tint of $\text{night} \times 0.20$ over entities. When night is above 0.25, the drone draws a soft radial-gradient headlight of radius 170 px, so darkness has a gameplay-visible effect.

Parallax. Distant objects appear to move more slowly than near ones. Each layer has a parallax factor and a repeating tilew; its offset is $-(\text{scrollX} \cdot \text{parallax}) \% \text{tilew}$, and the layer is drawn repeatedly from one tile before the screen edge to one after it, giving seamless infinite scrolling.

Layer	Parallax factor	Tile width	Content
Far ridge	0.055	1600 px	Pale mountain outline
Near ridge	0.130	1280 px	Closer hills
Far trees	0.300	1100 px	Small acacia and baobab silhouettes
Near trees	0.520	900 px	Large, dark silhouettes; road dashes and ground detail scroll at 1.0

Procedural terrain. Ridge outlines are the sum of three sine waves with frequencies in the ratio 1 : 2.3 : 5.7 and amplitudes 1 : 0.42 : 0.14, with a random phase from a seeded generator (mulberry32). Using non-integer ratios prevents visible repetition inside a tile, and using fixed seeds means the landscape is identical every run, which keeps testing and debugging repeatable. Trees, stars, and ground detail are placed the same way.

5.5 Score system and local storage (2.4)

Scoring event	Points
Distance flown	1 point per metre (pointPerMetre)
Cargo pod collected	50 points (cargoPoints)
Delivery completed (every 5 pods)	250 bonus points (deliveryBonus) plus a celebratory sound and particle burst
Obstacle safely passed	15 points and a horn

Energy-efficiency score. Efficiency is $\text{distance} / \text{totalDrain}$, i.e. metres travelled per percent of battery consumed. It is shown live on the HUD and saved with each run, rewarding pilots who use recharge zones well and avoid needless thrusting.

Local storage. On game over, a run record (score, distance, cargo, deliveries, efficiency, reason, and ISO date) is added to the stored list, the list is sorted by score, and only the best five are kept under the key `ecodash.scores.v1`. Preferences such as mute are stored separately under `ecodash.prefs.v1`. All reads and writes are wrapped in try/catch so private-browsing or quota errors cannot crash the game (Mozilla, n.d.c).

```

saveScore: function (entry) {
  var list = EcoDash.Storage.getScores();
  list.push(entry);
  list.sort(function (a, b) { return b.score - a.score; });
  list = list.slice(0, MAX_ENTRIES);    // keep top 5
  try { localStorage.setItem(EcoDash.Config.storageKey, JSON.stringify(list)); }
  catch (e) {}
  return list;
}

```

Game states and restart. The start screen, pause screen (P or Esc, Resume and Restart buttons) and game-over screen (six statistic tiles with the reason for failure, and a "Fly Again" button) are DOM overlays toggled by state. `resetRun()` rebuilds the drone, clears entities and particles, and resets score, timers, load-shedding, and weather, so a new mission starts without a page reload. A short lock-out after a crash prevents accidental instant restarts.

6. Testing, known limitations, and code-defence notes

6.1 Test plan

Each test was run with scripted inputs against the game logic, and the observed result is recorded below. Items marked Partial have a visual or audio part that is confirmed by eye in the browser.

#	Test	Expected result	Result	Notes
1	Press Space on the start screen	Mission starts, music plays, HUD appears	Partial	State menu to playing. Confirm music and HUD by eye.
2	Hold thrust, then release	Drone climbs, then falls smoothly; rotors speed up while held	Pass	Held: y 288 to 242. Released: falls. Rotors 44 vs 26 rad/s.
3	Fly into a pylon	Screen shake, explosion, game-over screen with reason "crash"	Pass	Reason "crash"; shake 0.96; debris burst.
4	Let the drone touch the ground	Mission ends with reason "ground"	Pass	Reason "ground".
5	Collect five pods	Delivery counter increases, +250 bonus, delivery jingle	Pass	5 pods = 1 delivery; +250 bonus.
6	Fly through a solar zone (grid on)	Battery rises quickly, sparkle particles, charge sound	Partial	Battery 30 to 57 % in 0.33 s. Confirm sparkles and sound by eye.
7	Fly through a solar zone during load-shedding	Zone is greyed and offline; battery does not rise	Pass	Battery 30 to 29.7 %; no recharge while grid off.
8	Let the battery reach 0	Mission ends with reason "battery"	Pass	Reason "battery".
9	Press P, then P again	Pauses (world frozen), then resumes	Pass	World frozen while paused; moves again on resume.
10	Crash, reload the page	Best score is still shown (localStorage)	Pass	Best score 1 234 read back by a new Game instance.
11	Press R after crashing	New mission starts without a page reload	Pass	Ignored in 0.85 s lock-out; works after; entities cleared.

#	Test	Expected result	Result	Notes
12	Resize the window or rotate the phone	Canvas and HUD rescale correctly	Partial	390x844: HUD scale 0.72; 1920x1080: 1.35; pixel ratio capped at 2. Confirm layout by eye.

6.2 Known limitations and future work

- The obstacle set covers wildlife (birds), power infrastructure, trees, construction, and storms; potholes and flooded rivers are not yet modelled, because the drone flies above the road. They would suit a ground-vehicle mode.
- Load-shedding is a global timer event, not tied to individual zones or areas; a future version could switch individual grid regions on and off.
- Rain reduces visibility through a tint and vignette but does not yet limit how far ahead obstacles can be seen.
- Only vertical control is provided (the world scrolls horizontally), and touch input provides thrust only.
- Delivery has no drop-off location or time limit; deliveries are counted from pods collected.

6.3 Code-defence quick reference

Common live-coding requests map to a single constant or function:

Requested change	Where to change it
Increase obstacle or world speed	Config.world.scrollSpeedStart / scrollSpeedMax; bird and storm extra speed in Bird and StormCell constructors
Change battery consumption	Config.player.drainBase, drainThrust, drainWeather, solarRegen
Adjust movement physics	Config.player.gravity, thrust, dive, drag, maxVy
Modify collision behaviour	Game.testCollision; drone radius in Config.player.radius
Add a keyboard shortcut	Game.handleGlobalKeys (use this.input.pressed("KeyX"))
Change load-shedding frequency	Config.loadShedding (delays and durations)
Change day length	CYCLE_SECONDS in world.js

7. Conclusion

EcoDash turns two real African infrastructure pressures, last-mile medical delivery and load-shedding, into a playable simulation whose behaviour is driven by explicit mathematics: kinematics with drag, sine-based wind and arcs, energy budgets, and circle and rectangle collision tests. The layered, object-oriented code, the procedural audio and the original day/night and parallax system show the animation techniques covered in WAS262.

References

- Daily Investor (n.d.) Eskom stage 7 and 8 load-shedding confusion. Available at: <https://dailyinvestor.com/energy/19191/eskom-stage-7-and-8-load-shedding-confusion/> (Accessed: 23 September 2026).
- Ericson, C. (2005) Real-time collision detection. San Francisco: Morgan Kaufmann.
- Flanagan, D. (2020) JavaScript: the definitive guide. 7th edn. Sebastopol: O'Reilly Media.
- Gregory, J. (2018) Game engine architecture. 3rd edn. Boca Raton: CRC Press.
- Halliday, D., Resnick, R., and Walker, J. (2014) Fundamentals of physics. 10th edn. Hoboken: Wiley.
- Mozilla (n.d.a) Canvas API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API (Accessed: 23 September 2026).
- Mozilla (n.d.b) Web Audio API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API (Accessed: 23 September 2026).
- Mozilla (n.d.c) Window: localStorage property. Available at: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage> (Accessed: 23 September 2026).
- Mozilla (n.d.d) Window: requestAnimationFrame() method. Available at: <https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame> (Accessed: 23 September 2026).
- NPR (2026) They're a world leader in using drones for healthcare. But drones can't do it all. 4 August. Available at: <https://www.vpm.org/npr-news/2026-08-04/theyre-a-world-leader-in-using-drones-for-healthcare-but-drones-cant-do-it-all> (Accessed: 23 September 2026).
- Nystrom, R. (2014) Game programming patterns. Genever Benning.
- STADIO (2026) WAS262 SF1 assignment instructions: EcoDash African Digital Logistics and Infrastructure Simulator. STADIO School of Information Technology.
- Thomas, F. and Johnston, O. (1981) The illusion of life: Disney animation. New York: Abbeville Press.